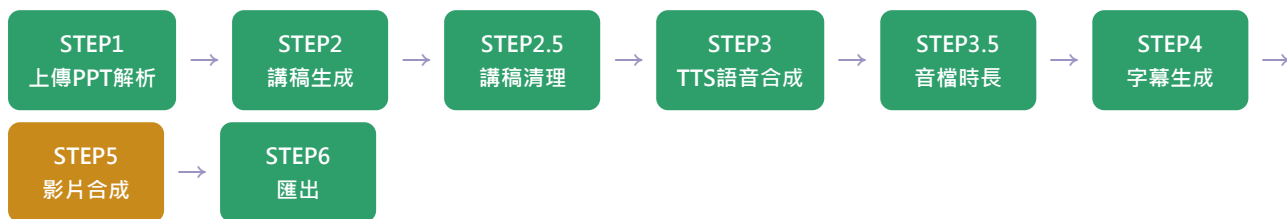


PPT2Course AI — 系統架構文件

PPT → Script → Script Cleaner → TTS → Subtitle → Video → Export | Python 3.14.6 | 108 個測試 | github.com/Kuo1204/ppt2course-v2-

1. 整體流程與完成狀態



完成 STEP1、2、2.5、3、3.5、4、6

核心完成 STEP5(轉場+字幕燒錄;Logo/BGM/片頭片尾未做)

目前沒有的東西：把六個 STEP 串起來的 pipeline 協調層。每個模組都獨立可用、有測試，但還沒有一個「丟一個 .pptx 進去、吐出一支影片」的總入口函式。

2. 模組總覽

STEP	檔案	核心函式 / 類別	職責
1	<code>upload.py</code>	<code>parse_ppt() → list[SlideContent]</code>	用 python-pptx 解析每張投影片的文字(文字框+表格，依畫面位置排序)與備忘稿
2	<code>script_gen.py</code>	<code>generate_script(mode, slides, texts)</code>	NOTES/OWN/AUTO/POLISH 四模式；AUTO/POLISH 是 passthrough(見第4節)
2.5	<code>script_cleaner.py</code>	<code>clean_script(text) → str</code>	去除空白行、分隔線、項目符號、Markdown 符號，壓縮多餘空白
3	<code>tts.py</code>	<code>synthesize(text, voice, out_path) → list[TimedChunk]</code>	呼叫 edge-tts，寫出音檔，並將 WordBoundary 事件與原文對齊，補回標點的零時長時間戳
3.5	<code>audio_duration.py</code>	<code>get_audio_duration_ms(path) → int</code>	呼叫 ffprobe 量測音檔長度(向下取整為毫秒)，時間軸的唯一依據
4	<code>subtitle.py</code>	<code>generate_cues() / cues_to_srt()</code>	把 TimedChunk 依標點/字數規則切成字幕行，產生標準 SRT
5	<code>video.py</code>	<code>compose_video(slides, out_video, out_srt, ...)</code>	xfade轉場+字幕燒錄，內部計算轉場後的累積時間軸並產生對應SRT
6	<code>export.py</code>	<code>export_outputs(mp4, srt, scripts, out_dir, base_name)</code>	輸出 {base}.mp4 / {base}.srt / {base}.docx 到指定資料夾

3. 核心資料結構

```
SlideContent(index: int, text: str, notes: str)           # STEP1 輸出
TimedChunk(text: str, start_ms: int, end_ms: int)        # STEP3 輸出／STEP4 輸入
SubtitleCue(index: int, start_ms: int, end_ms: int, text: str) # STEP4 輸出
SlideVideoInput(image_path: str, audio_path: str, chunks: list[TimedChunk]) # STEP5 輸入
```

`TimedChunk` 的 `text` 可能是多個字元(例如「世界」)——這是 STEP3 開發時實測 edge-tts 得到的真實行為，見第4節。

4. 關鍵架構決策

4.1 edge-tts 的真實行為 → STEP4 從「逐字元」改為「原子區塊」

實測發現 edge-tts 的 `WordBoundary` 事件並非嚴格逐字元(有時把「世界」合成一個事件)，且標點符號完全沒有自己的時間戳。因此：

- `CharTiming` 改名為 `TimedChunk`，一個區塊可能是多字元，且視為**不可拆分**的原子單位
- STEP4 的斷行邏輯改用字元數加總(而非區塊數)判斷 15 字上限，切點只能落在區塊邊界
- STEP3 的 `_align_with_original_text()` 負責把 edge-tts 跳過的標點符號補回完整序列：標點的 `start_ms/end_ms` 都等於前一個區塊的 `end_ms`(零時長)

4.2 STEP5 轉場時間軸：三者同步保證

`compose_video()` 是唯一計算「轉場後累積時間軸」的地方，避免影片與字幕各自算出不同的偏移量。

```
offsets[0] = 0
offsets[i] = offsets[i-1] + durations[i-1] - transition_duration_ms
```

xfade(影片)與 acrossfade(音訊)都會重疊 `transition_duration`，所以總長度 = Σ 音檔長度 - $(N-1) \times$ 轉場時長。

額外發現並解決的問題：轉場重疊期間，前後兩張投影片的旁白音檔會同時播放。若直接把所有投影片的 `TimedChunk` 攤平丟給 STEP4 的 `generate_cues()`，其既有的「向前修剪」間隔邏輯會不當地截短前一張片還沒講完的字幕。解法：**每張投影片分開呼叫 `generate_cues()`**，STEP5 自行做「下一張整批延後、絕不截斷前一張」的銜接處理，讓 STEP4 原本針對單一音軌設計的邏輯保持不變。

4.3 AUTO / POLISH：AI 呼叫在瀏覽器端，不在 Python 後端

使用者將自行取得 Gemini API Key 並在網頁上輸入；瀏覽器直接用該 key 呼叫 Gemini API，金鑰從不經過這個 Python 後端。因此 STEP2 的 `generate_script()` 對 `AUTO` / `POLISH` 模式而言，行為等同 `OWN` (接收已生成好的文字、原樣回傳)，差別僅在於：AI 產出的文字不允許空字串(視為 AI 呼叫失敗)，`OWN` 則允許(代表使用者刻意留白、不要旁白)。

4.4 字幕斷行規則(STEP4)

規則	值
每行最大字數	15 字(區塊視為原子，極端情況下單一超長區塊可獨佔一行並超過上限)
強制斷行	。 ！ ？ (無條件斷行，即使因此讓行末剛好卡在16字也接受)
條件斷行	， 、 ； m僅在超過字數上限時才作為斷點(word-wrap 風格，優先用最近的斷點)
最短時長	1 秒，不足則與下一 cue 合併(合併後不得超過15字)
cue 間隔	保留 100ms 最小間隔
行末標點	保留 ？ ！ ； 去除 。 ， 、 ；

5. 外部依賴與測試策略

依賴	用途	測試方式
python-pptx	STEP1 解析 .pptx	用 python-pptx 自己動態產生測試用 .pptx，無須 mock
edge-tts	STEP3 呼叫 Microsoft 免費 TTS 服務	Mock <code>Communicate.stream()</code> 做單元測試 + 真實網路整合測試
ffprobe	STEP3.5 量測音檔長度	Mock <code>subprocess.run</code> + 真實 ffprobe 整合測試
ffmpeg	STEP5 影片合成(xfade/acrossfade/字幕燒錄)	Mock 驗證指令組裝 + 真實產生短影片的整合測試
python-docx	STEP6 產生講稿.docx	寫入後用 python-docx 讀回驗證內容

共 108 個測試，全部通過。每個牽涉外部服務/程式的模組都採用「mock 驗證邏輯 + 真實呼叫驗證端到端」雙軌策略；真實整合測試曾兩度抓到 mock 測試無法發現的 bug(STEP5 的 `[a0]/[a1]` 音訊標籤未定義；STEP3 的 WordBoundary 粒度假設錯誤)。

6. 明確排除在目前範圍外

- STEP5：Logo 疊加、BGM 混音、片頭片尾(側邊註記中的「選填」項目)
- 整個網頁前端(含使用者輸入 Gemini API Key 的介面)
- Pipeline 協調層 — 目前沒有一個函式把 STEP1~6 依序串起來執行